

VHDL Programming

LAB: Digital Electronics

VHDL

- VHDL stands for VHSIC Hardware Descriptive Language., where VHSIC (Very High Speed Integrated Circuits)
- Developed in the mid-1980s by U.S. Department of Defense (DoD) and IEEE
- The other widely used hardware description language is Verilog .
- Both are powerful languages that allow you to describe and simulate digital systems.
- A third HDL language is ABEL (Advanced Boolean Equation Language) which was specifically designed for Programmable Logic Devices (PLD).

VHDL Coding Tool: Simulation

- **Text editor:**

VHDL is a text-based language. A specialized VHDL text editor include features like automatic highlighting of VHDL keywords, automatic indenting, built-in templates for frequently used program structures, and built-in syntax checking and one-click access to the compiler.

- **Compiler:**

A VHDL compiler analyzes code for syntax errors.. Facilitate insertion of breakpoint to run piece of code. It also creates the internal information that is needed for a simulator to process your design.

- **Simulator:**

A VHDL simulator allows you to define and apply inputs to your design, and to observe its outputs, without ever having to build the physical circuit.

VHDL Coding: Verification

VHDL environment gives ability to create “test benches” that automatically apply inputs to compare circuit response with expected outputs.

- Verification type

There are two types of verification

1) Functional Verification

2) Timing Verification

- In functional verification, we study the circuit’s logical operation independent of timing considerations; gate delays and other timing parameters are considered zero.
- In timing verification, we study the circuit’s operation including estimated delays, and we verify that the setup, hold, and other timing requirements for sequential devices like flip-flops are met.

* Verification process associated with certain back-end steps.

VHDL Coding : Synthesis

In back-end there are three basic steps

- 1) Synthesis**
- 2) Fitting/Place + Route**
- 3) Timing Verification**

- **Synthesis:**

Synthesis convert the VHDL description into a set of primitives or components that can be assembled in the target technology. For example, with PLDs or CPLDs, the synthesis tool may generate two-level sum-of-products equations.

- **Fitting/Place + Route**

In the fitting step, a fitting tool or fitter maps the synthesized primitives or components onto available device resources. For a PLD or CPLD, it assign equations to available AND-OR elements.

- **Timing Verification**

The “final” step is timing verification of the fitted circuit. Actual circuit delays due to wire lengths, loading and other factors can be calculated with reasonable precision.

VHDL Program Structure

A VHDL Program consists of

- **entity** part: It is simply a declaration of a module's inputs and outputs
- **Architecture** part: A detailed description of the module's internal structure or behavior.

Syntax of a VHDL entity declaration.

```
entity entity-name is
    port (signal-names : mode signal-type;
          signal-names : mode signal-type;
          ...
          signal-names : mode signal-type);
end entity-name;
```

*The entity declaration is to define its external interface. **entity, is, port, and end** are the keywords

*Note that there is no semicolon after the final signal-type instead closing parenthesis with the semicolon after it.

Entity declaration

- **entity-name:** A user-selected identifier to name the entity
- **signal-names :** A comma-separated list of one or more user-selected identifiers to name external-interface signals.
- **mode**
One of four reserved words, specifying the signal direction
 - in :** The signal is an input to the entity.
 - out:** The signal is an output of the entity. Note that the value of such a signal cannot be “read” inside the entity’s architecture, only by other entities that use it.
 - buffer:** The signal is an output of the entity, and its value can also be read inside the entity’s architecture.
- **inout :** The signal can be used as an input or an output of the entity. This mode is typically used for three- state input/output pins on PLDs.

```
entity entity-name is
    port (signal-names : mode signal-type;
          signal-names : mode signal-type;
          ...
          signal-names : mode signal-type);
end entity-name;
```

Entity declaration

- signal-type : A built-in or user-defined signal type.
- **built-in signal type.** Examples of types are bit, bit_vector, Boolean, character, std_logic, and std_ulogic.

Description as follows

- bit – can have the value 0 and 1
- bit_vector – is a vector of bit values (e.g. bit_vector (0 to 7))
- std_logic, std_ulogic, std_logic_vector, std_ulogic_vector: can have 9 values to indicate the value and strength of a signal. Std_ulogic and std_logic are preferred over the bit or bit_vector types.
- boolean – can have the value TRUE and FALSE.
- integer – can have a range of integer values.
- real – can have a range of real values.
- character – any printing character.
- time – to indicate time.

Architecture Declaration

The architecture body specifies how the circuit operates and how it is implemented. An entity or circuit can be specified in a variety of ways, such as

- Data flow
- behavioral,
- structural (interconnected components),
- or a combination of the above.

The entity-name during architecture declaration must be the same as in the entity declaration.

architecture body

- The architecture body looks as follows,

architecture **architecture_name** of **entity-name** is

Set of Declarations

-- components declarations

-- signal declarations

-- constant/ variable declarations

signal **signal-names** : **signal-type**;

Variable **variable-names** : **variable-type**;

constant **name**: **type-name** := **value**

ex: constant BUS_SIZE: integer := 32;

-- function declarations

-- procedure declarations

-- type declarations

begin

-- Statements

-- Statements

--

end **architecture_name**

Architecture style/ modeling

Data Flow Modeling

- In this modeling style, **the flow of data through the entity is expressed using concurrent (parallel) signal.**
- The concurrent statements in VHDL are **When-else, With-select and Generate .**
- Besides them, assignments using only **operators (AND, NOT, +, * etc.)** can also be used to construct the code.

VHDL program structure

1. Define Library functions.
2. Declare Entity and Architecture.
3. Describe functionality.
4. End source code.
5. Compile and Run program.

VHDL code for Logic gates

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

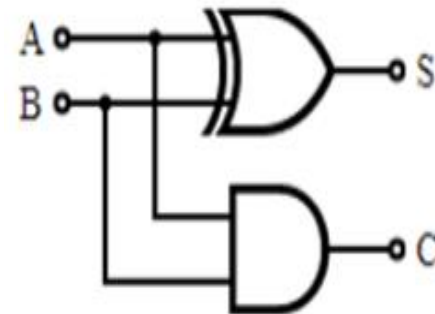
entity ALLGATES_SOURCE is
Port ( A,B : in  STD_LOGIC;
      P, Q, R, S, T, U, V : out STD_LOGIC);
end ALLGATES_SOURCE;
architecture dataflow of
    ALLGATES_SOURCE is
begin
P <= A and B;
Q <= A nand B;
R <= A or B;
S <= A nor B;
T <= not A;
U <= A xor B;
V <= A xnor B;
end dataflow;
```

VHDL Code Half adder

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.STD_LOGIC_ARITH.ALL;  
use  
    IEEE.STD_LOGIC_UNSIGNED.ALL  
    ;
```

```
entity half_adder is  
port(a,b:in bit; sum,carry:out bit);  
end half_adder;
```

```
architecture data of half_adder is  
begin  
sum<= a xor b;  
    carry <= a and b;  
end data;
```



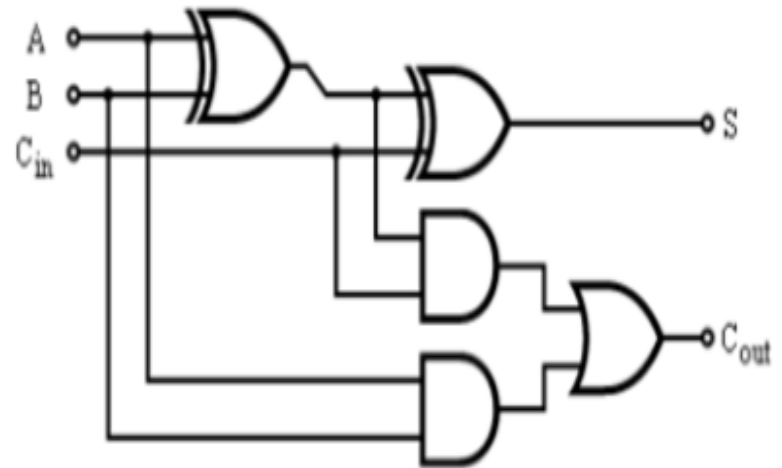
Half-Adder Schematic

VHDL Code Full adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity full_adder is
port (a,b,c:in bit;
sum,carry:out bit);
end full_adder;

architecture data of full_adder is
begin
sum<= a xor b xor c;
carry <= ((a and b) or (b and c) or (a and
      c));
end data;
```



Full-Adder |

4 to 1 Mux data flow model (with-select statement)

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.STD_LOGIC_ARITH.ALL;  
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
entity MUX4_1 is  
  Port ( i : in  STD_LOGIC_VECTOR (3 downto 0);  
        s : in  STD_LOGIC_VECTOR (1 downto 0);  
        y : out STD_LOGIC);  
end MUX4_1;
```

architecture dataflow of MUX4_1 is

begin

with s select

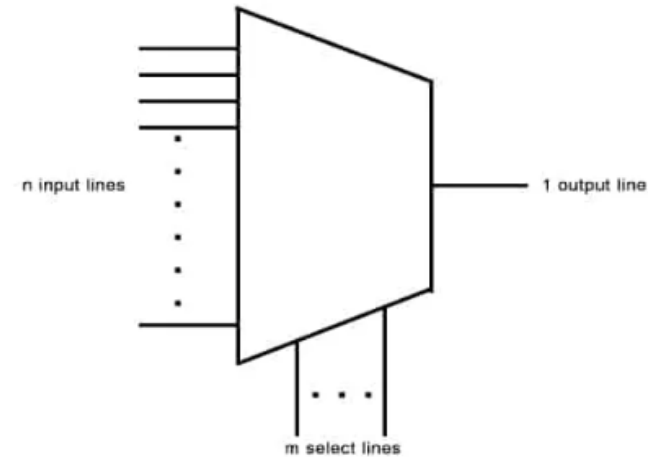
y <= i(0) when "00", --- commas at the end of each select output
assignment and a semicolon to close off the last statement

i(1) when "01",

i(2) when "10",

i(3) when others;

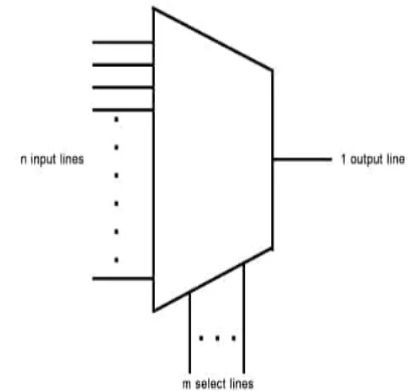
end dataflow;



S0	S1	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

4 to 1 Mux data flow model (When-else statement)

- **library ieee;**
- **use ieee.std_logic_1164.all;**
- **entity multiplexerEx is**
- **port(s: in std_logic_vector(1 downto 0);**
- **i0, i1, i2, i3: in std_logic;**
- **y: out std_logic);**
- **end multiplexerEx;**
- **architecture dataflow of multiplexerEx is**
- **begin**
- **y <= i0 when s = "00" else**
- **i1 when s = "01" else**
- **i2 when s = "10" else**
- **i3 when s = "11";**
- **end dataflow;**



S0	S1	Y
0	0	i0
0	1	i1
1	0	i2
1	1	i3

Dataflow model

- ‘Generate statement’ is used for creating loops in concurrent assignments. These loops are very different from software loops.
- Suppose ‘for i = 1 to N’ is a loop, then, in software ‘i’ will be assigned one value at a time i.e. first i=1, then next cycle i=2 and so on. Whereas in VHDL, N logic will be implement for this loop, which will execute in parallel.
- Also, in software, ‘N’ cycles are required to complete the loop, whereas in VHDL the loop will execute in one cycle.

Practice assignment

Write VHDL code with dataflow model for the following:

- Half subtractor
- Full subtractor
- Comparator
- Demultiplexer
- Encoder
- Decoder